

# Route5 概率计算与 SDP 构造说明

本文档是对 `guessprobprimal_route5_hybrid_iq.m` 的一个专题补充说明，专门集中解释两件事：

- route5 的离散概率表  $P(c|x, y)$  是怎么从相干态输入和 IQ 测量模型一步步算出来的；
- 正式认证时使用的 single-device guessing SDP 是如何构造的、优化变量是什么、目标函数和约束分别代表什么物理意义。

这份小报告与主报告 `路线5的理论实验分析报告` 的关系是：

- 主报告负责给出完整理论主线、程序流程、实验接口和背景解释；
- 本文档只聚焦“概率”和“SDP”两部分，便于导师或同学快速查阅。

## 1. 问题的最小抽象

route5 的核心流程可以压缩成下面这条链：

trusted coherent inputs  $\rightarrow$  beam splitter + dual-homodyne  $\rightarrow$  IQ coarse-graining  $\rightarrow P(c|x, y) \rightarrow$  guessing SDP  $\rightarrow p_{\text{guess}}, H_{\min}$ .

其中：

- $x, y$  表示 Alice 和 Bob 两边选择的本地输入标签；
- $c$  表示中央测量的离散输出标签；
- $P(c|x, y)$  是进入 SDP 的唯一统计对象；
- 正式最小熵由

$$H_{\min} = -\log_2 p_{\text{guess}}$$

给出。

所以从数学上说，route5 可以分成两段：

- 先从物理模型生成概率表  $P(c|x, y)$ ；
- 再把这张概率表连同 trusted input states 一起送入 SDP。

## 2. 概率计算部分

### 2.1 本地输入态

设本地 coherent alphabet 为

$$\mathcal{A} = \{\alpha_\ell\}_{\ell=1}^L.$$

对每个复振幅  $\alpha_\ell$ ，构造截断 Fock 空间中的相干态

$$|\alpha_\ell; d\rangle = \frac{1}{\sqrt{\mathcal{N}_\ell}} \sum_{n=0}^{d-1} e^{-|\alpha_\ell|^2/2} \frac{\alpha_\ell^n}{\sqrt{n!}} |n\rangle,$$

其中  $d = \text{cutoff}$ ， $\mathcal{N}_\ell$  是截断后的归一化因子。

对应密度矩阵为

$$\rho_\ell = |\alpha_\ell; d\rangle \langle \alpha_\ell; d|.$$

程序中这一步由 Matlab 文件中的相干态构造函数完成；在 Python 主线中对应 `hybrid_iq.py` 里的 `alphabet` 构造与态生成逻辑。

## 2.2 支持压缩

虽然一开始所有态都写在  $d$  维截断空间里，但这些态张成的真实支持子空间通常更小。设本地支持基为

$$B_{\text{loc}} \in \mathbb{C}^{d \times r}, \quad r \leq d,$$

则压缩后的本地态为

$$\tilde{\rho}_\ell = B_{\text{loc}}^\dagger \rho_\ell B_{\text{loc}}.$$

这一步不改变物理预测，只是把后续计算搬到更小的有效子空间中。

## 2.3 联合输入态

对 Alice 选择  $x$ 、Bob 选择  $y$ ，联合输入态为

$$\tilde{\rho}_{xy} = \tilde{\rho}_x \otimes \tilde{\rho}_y.$$

因此总输入数为

$$N_{\text{in}} = L^2.$$

在当前强点主线中，典型参数是 `num_local_states = 17`，所以联合输入总数是

$$17^2 = 289.$$

## 2.4 中央连续测量：分束器加 dual-homodyne

中央首先对双模输入施加平衡分束器酉变换

$$U_{\text{BS}} = \exp \left[ \frac{\pi}{4} (a^\dagger b - ab^\dagger) \right].$$

然后一路做  $X$  测量，一路做  $P$  测量。因此理想连续 POVM 形式为

$$M(x, p) = U_{\text{BS}}^\dagger (|x\rangle\langle x| \otimes |p\rangle\langle p|) U_{\text{BS}}.$$

这一步对应的是连续 IQ 输出  $(x, p)$ ，但实际认证并不直接用连续输出，而是继续做 coarse-graining。

## 2.5 IQ 分箱

设  $X$  方向边界为

$$-\infty = b_0^{(x)} < b_1^{(x)} < \dots < b_{N_x}^{(x)} = +\infty,$$

设  $P$  方向边界为

$$-\infty = b_0^{(p)} < b_1^{(p)} < \dots < b_{N_p}^{(p)} = +\infty.$$

则第  $(i, j)$  个输出 bin 对应矩形区域

$$R_{ij} = [b_{i-1}^{(x)}, b_i^{(x)}) \times [b_{j-1}^{(p)}, b_j^{(p)}).$$

总输出数为

$$C = N_x N_p.$$

例如当前最强主线常用

$$N_x = 6, \quad N_p = 2, \quad C = 12.$$

## 2.6 单模 quadrature POVM 的数值构造

对单模相位为  $\theta$  的 quadrature

$$\hat{X}_\theta = \frac{1}{\sqrt{2}} (ae^{-i\theta} + a^\dagger e^{i\theta}),$$

第  $k$  个单模 bin 的理想 POVM 元是

$$F_k^{(\theta)} = \int_{I_k} |x_\theta\rangle\langle x_\theta| dx.$$

Matlab 脚本并不解析地算这个积分，而是使用高斯-厄米特求积近似：

$$F_k^{(0)} \approx \sum_{j=1}^K m_{k,j} w_j |x_j\rangle\langle x_j|,$$

其中：

- $x_j$  是求积节点；
- $w_j$  是求积权重；
- $m_{k,j} \in \{0, 1\}$  表示节点  $x_j$  是否落在第  $k$  个 bin 中。

把它写到 Fock 基矩阵元上，可以得到

$$[F_k^{(0)}]_{nm} \approx \sum_{j=1}^K \varphi_n(x_j) \varphi_m(x_j) w_j m_{k,j},$$

其中  $\varphi_n(x)$  是 Hermite 函数。

然后利用旋转关系

$$[F_k^{(\theta)}]_{nm} = e^{-i\theta(n-m)} [F_k^{(0)}]_{nm},$$

得到任意相位  $\theta$  的 quadrature POVM。

## 2.7 POVM 完备性修正

由于数值积分和截断误差，直接求得的 POVM 元求和一般只有

$$S = \sum_k F_k^{(\theta)} \approx I,$$

而不严格等于  $I$ 。因此脚本再做一次白化修正：

$$\tilde{F}_k^{(\theta)} = S^{-1/2} F_k^{(\theta)} S^{-1/2}.$$

修正后满足

$$\sum_k \tilde{F}_k^{(\theta)} = I.$$

这一步不是在改物理模型，而是在把数值近似拉回严格 POVM。

## 2.8 双模离散 IQ POVM

设  $X$  方向 coarse POVM 为  $\{\tilde{F}_i^{(x)}\}$ ， $P$  方向 coarse POVM 为  $\{\tilde{F}_j^{(p)}\}$ ，则中央离散输出  $(i, j)$  的双模效应为

$$E_{ij} = U_{\text{BS}}^\dagger \left( \tilde{F}_i^{(x)} \otimes \tilde{F}_j^{(p)} \right) U_{\text{BS}}.$$

之后再把它投影到联合支持子空间：

$$\tilde{E}_{ij} = B_{\text{joint}}^\dagger E_{ij} B_{\text{joint}}.$$

## 2.9 最终 Born 概率

到这里，最终进入 SDP 的概率表就是

$$P(c|x,y) = \text{Tr}\left(\tilde{E}_c \tilde{\rho}_{xy}\right).$$

这里  $c$  只是把二维标签  $(i,j)$  压成一个一维输出编号后的写法。

程序上，对所有输入  $(x,y)$  和所有输出  $c$  逐项计算上式，就得到完整概率矩阵

$$\mathbf{P} \in \mathbb{R}^{L^2 \times C}.$$

这就是 `probabilities` 。

## 2.10 Raw 熵只是预筛选

对每个输入  $s = (x,y)$ ，定义 raw 猜测概率

$$p_{\text{guess}}^{\text{raw}}(s) = \max_c P(c|s),$$

对应 raw 熵

$$H_{\min}^{\text{raw}}(s) = -\log_2 p_{\text{guess}}^{\text{raw}}(s).$$

需要特别强调：

$$H_{\min}^{\text{raw}} \neq H_{\min}^{\text{formal}}.$$

`raw_H_min` 只是在正式 SDP 之前用来挑目标输入的便宜指标，它不是最终安全结论。

# 3. SDP 构造部分

## 3.1 SDP 的任务是什么

概率表  $P(c|x,y)$  算出来以后，真正的问题就变成：

对某个目标输入  $s^*$ ，在所有与这些统计兼容的量子实现里，Eve 最多能把输出猜多准？

这个“最坏情况下的最佳猜中率”就是

$$p_{\text{guess}}(s^*).$$

一旦得到它，就可以定义正式最小熵

$$H_{\min}(s^*) = -\log_2 p_{\text{guess}}(s^*).$$

## 3.2 优化变量

对每个真实输出  $c \in \{1, \dots, C\}$  和 Eve 的猜测  $e \in \{1, \dots, C\}$ ，引入一个厄米半正定算符

$$M_{c,e} \succeq 0.$$

这些变量可以理解为一个“真实输出 / Eve 猜测”双索引表。

在 Matlab 脚本里，它通过三维数组变量实现：

```
variable M_ops(dimension, dimension, num_outputs * num_outputs) hermitian semidefinite
```

其中每一片 `M_ops(:, :, op_idx)` 对应某个  $M_{c,e}$ 。

### 3.3 目标函数

若认证目标输入是  $\rho_{s^*}$ ，则 Eve 猜中的概率只来自“真实输出等于猜测输出”的那些对角块，于是目标函数为

$$p_{\text{guess}}(s^*) = \max \sum_{c=1}^C \text{Tr}(M_{c,c} \rho_{s^*}).$$

这个结构非常重要，因为它说明 formal 认证并不是直接把  $\max_c P(c|s^*)$  再算一遍，而是在更大的量子兼容策略集合上取最坏情况。

### 3.4 统计一致性约束

对每个输入  $s$  和每个真实输出  $c$ ，都要求

$$\sum_{e=1}^C \text{Tr}(M_{c,e} \rho_s) = P(c|s).$$

这条约束的物理含义是：

- 不管 Eve 在内部怎么分解策略；
- 不管她最终准备猜哪个  $e$ ；
- 对外表现出来的真实统计必须仍然和观测概率表完全一致。

所以 route5 的 formal 安全性始终被钉在同一张 probabilities 表上。

### 3.5 完备性约束

对每个猜测标签  $e$ ，要求

$$\sum_{c=1}^C M_{c,e} = p_e I,$$

其中

$$p_e \geq 0, \quad \sum_{e=1}^C p_e = 1.$$

这表示：

- 每个固定的  $e$  都对应一套合法测量分支；
- 所有分支再按概率权重  $p_e$  混合；
- 这正是 single-device guessing SDP 的标准可实现结构。

### 3.6 半正定约束

每个变量块都必须满足

$$M_{c,e} \succeq 0.$$

这保证每个块都代表合法的正算符。

### 3.7 完整 SDP 写法

综合起来，对固定目标输入  $s^*$ ，脚本求解的是

$$\begin{aligned}
& \max_{\{M_{c,e}\}, \{p_e\}} && \sum_{c=1}^C \text{Tr}(M_{c,c} \rho_{s^*}) \\
& \text{s.t.} && M_{c,e} \succeq 0, \quad \forall c, e, \\
& && \sum_{e=1}^C \text{Tr}(M_{c,e} \rho_s) = P(c|s), \quad \forall s, c, \\
& && \sum_{c=1}^C M_{c,e} = p_e I, \quad \forall e, \\
& && p_e \geq 0, \quad \forall e, \\
& && \sum_{e=1}^C p_e = 1.
\end{aligned}$$

求得最优值  $\hat{p}_{\text{guess}}$  后，输出

$$\hat{H}_{\min} = -\log_2 \hat{p}_{\text{guess}}.$$

### 3.8 为什么这不是简单的线性规划

如果只看离散概率表，很容易误以为 formal 认证只是某种 classical post-processing。但这里的变量是算符  $M_{c,e}$ ，而不是标量，所以问题本质上仍是一个量子兼容性优化问题。

也正因此，formal H\_min 往往显著低于 raw H\_min。

例如当前最强 route5 点大致呈现：

$$H_{\min}^{\text{raw}} \approx 3.11, \quad H_{\min}^{\text{formal}} \approx 2.15.$$

中间接近 1 bit 的落差，正是 SDP 在把量子兼容的“最坏情况”算进去。

### 3.9 为什么脚本通常不认证所有输入

如果本地 alphabet 有  $L$  个态，则联合输入数为  $L^2$ 。当前强点里  $L = 17$ ，所以联合输入数是 289。如果对每个输入都做一次完整 SDP，计算成本很高。

因此脚本采用两阶段策略：

1. 先对所有输入算 raw 指标；
2. 再只对前  $K = \text{max\_inputs\_to\_certify}$  个候选输入做 formal 认证。

这个策略的好处是省算力，坏处是 raw 排名和 formal 排名不一定一致。所以在高精度搜索时，常常要扩大  $K$ ，或者像最近的 Python 搜索那样，直接固定强分区后对少量半径候选做 formal 逐点搜索。

## 4. SDP 是怎么求解的

Matlab 脚本中，这个问题通过 CVX + MOSEK 求解。

典型流程是：

1. 设置 cvx\_begin sdp
2. 声明变量 M\_ops 和 p\_e
3. 写入目标函数
4. 写入统计一致性、完备性和半正定约束
5. 调用 cvx\_end
6. 读取 cvx\_optval

最后返回：

```
result_struct.p_guess = cvx_optval;
result_struct.H_min = -log2(cvx_optval);
```

在 Python 主线中，相同理论问题由 CVXPY 封装并交给 MOSEK 、 SCS 或其他后端求解。

需要注意两点：

1. 求解器返回的 `optimal` / `optimal_inaccurate` / `solver_failed` 等状态会直接影响结果可报告性；
2. 对高维候选，求解成本通常主要由 SDP 部分承担，而不是概率生成部分。

## 5. 这两部分在程序中的分工

从程序结构上看，`route5` 可以非常清楚地拆成两层：

### 5.1 概率层

这一层负责生成

$$P(c|x,y).$$

它包含：

- 字母表构造；
- 支持压缩；
- 分束器；
- quadrature POVM 数值积分；
- IQ 分箱；
- Born 迹计算。

### 5.2 认证层

这一层负责计算

$$p_{\text{guess}}, H_{\min}.$$

它只需要两类输入：

- trusted input states  $\{\rho_s\}$ ；
- 概率表  $P(c|s)$ 。

这也是为什么 `route5` 将来如果接实验数据，只需要把“概率生成”那一层替换成实验概率表，而 SDP 层原则上可以保持不变。

## 6. 一句话总结

如果把整个 `route5` 压缩成一句话，那么它做的是：

先把固定的 coherent alphabet 通过 beam splitter 和 dual-homodyne 变成一张离散 IQ 概率表  $P(c|x,y)$ ，再用 single-device guessing SDP 计算在所有兼容量子策略中 Eve 的最优猜测概率，从而得到正式最小熵  $H_{\min}$ 。